Git init , reset and fork commands

Class no:4

What Does the git init Command Do?

The git init command is used to initialize a new Git repository in a directory. It creates a hidden .git folder in the specified directory, which contains all the metadata, configuration, and history required for Git to track changes in your project. This command is essential for starting version control in a project.

Key Features of git init

- **Creates a Git Repository**: Transforms the current directory into a Git repository by adding a .git folder.
- **Tracks Changes**: Enables Git to start tracking changes in files once they are staged and committed.
- **Works with Existing or New Projects**: Can be used in an empty directory or an existing project folder.
- **Foundation for Collaboration**: Allows linking to remote repositories (e.g., GitHub) for team collaboration.
- **Customizable Options**: Supports advanced configurations like creating bare repositories or using templates.

Key Differences Table

Feature	git init	git clone
Repository Type	Empty repository	Copies an existing repository
Remote Connection	Not created	Sets up origin automatically
Use Case	Start a new project	Work on an existing project
Commit History	None initially	Full commit history is cloned
Files	None unless manually added	All files from the remote repository

Summary:

- Use **git init** if you are starting a brand-new project from scratch.
- Use **git clone** if you want to get a complete copy of an existing repository to work on locally.

Hands on: Creating a repo in local using git init and add a remote repo to the local one:

Step no:01 Create a separate folder: **nithyafolder** in downloads.

Initialise .git folder in the local repo using: **git init**

Create a new file using **touch.**

Add it and make first commit.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~
$ cd downloads

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads
$ mkdir nithyafolder

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads
$ cd nithyafolder/

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder
$ git init
Initialized empty Git repository in C:/Users/nithya/Downloads/nithyafolder/.git/

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ touch app.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git add .

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git commit -m "first commit"
[main (root-commit) bf14698] first commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 app.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$
```

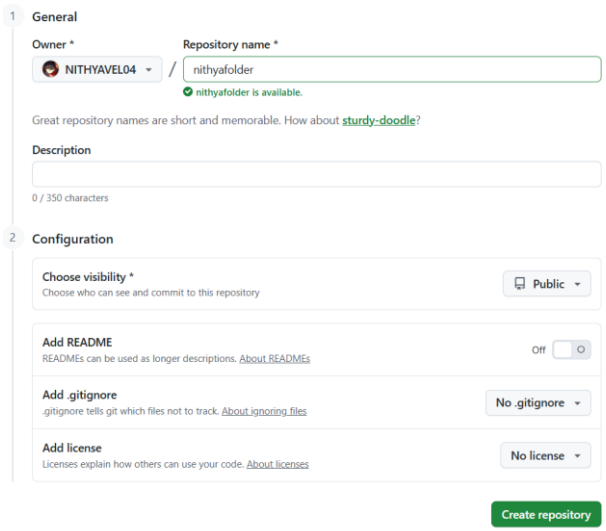
Rename the branch name from **master** to **main : git branch -M newname**

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git branch -M main
```

Step no:2 As we don't have any repository in remote to push this folder. Go to git hub and create a repo with same name as the folder name.

Ensure we are not making any changes, keep it default or else it will create a initial commit.

So just create a repo in remote without any changes.



Copy the repo url → git bash → add the remote origin to our local → push the local repo to that origin

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git remote add origin https://github.com/NITHYAVEL04/nithyafolder.git

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 219 bytes | 36.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/NITHYAVEL04/nithyafolder.git
 * [new branch]      main -> main
```

Check the remote repo available we can see the link of our remote repo here.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git remote -v
origin  https://github.com/NITHYAVEL04/nithyafolder.git (fetch)
origin  https://github.com/NITHYAVEL04/nithyafolder.git (push)
```

Git Reset HEAD Command

The git reset command is used to undo changes in a Git repository. It can modify the index (staging area), the working directory, and the commit history. The HEAD in Git refers to the current branch's latest commit. The git reset HEAD command is commonly used to unstage changes that have been added to the index but not yet committed.

Syntax and Usage

The basic syntax for the git reset command is:

git reset [<mode>] [<commit>]

Modes of git reset

- **--soft**: Moves the HEAD to the specified commit but does not change the index or working directory. The changes remain staged.
- **--mixed**: Moves the HEAD to the specified commit and updates the index to match, but does not change the working directory. This is the default mode.
- **--hard**: Moves the HEAD to the specified commit and updates both the index and working directory to match. All changes are discarded.

Step no:1 Add four files and make extra four commits.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git log --oneline
bf14698 (HEAD -> main, origin/main) first commit

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ touch file1.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git add .

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git commit "second commit"
error: pathspec 'second commit' did not match any file(s) known to g

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git commit -m "second commit"
[main 460f410] second commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file1.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ touch file2.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git add .

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git commit -m "third commit"
[main a185a80] third commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file2.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ touch file3.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git add .

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git commit -m "fourth commit"
[main 3732bde] fourth commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file3.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ touch file4.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git add .

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git commit -m "fifth commit"
[main c5cb36f] fifth commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file4.py
```

Step no:2 Check the commit history using **log –oneline**

As we can see the head is at the fifth commit.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git commit -m "fifth commit"
[main c5cb36f] fifth commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file4.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git log --oneline
c5cb36f (HEAD -> main) fifth commit
3732bde fourth commit
a185a80 third commit
460f410 second commit
bf14698 (origin/main) first commit
```

Open our existing **app.py** file and make any changes.

Check the status of the file. It is in untracked files.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ vi app.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ ls
app.py  file1.py  file2.py  file3.py  file4.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git status -s
 M app.py
```

I didn’t push from the second to fifth commit. I only pushed first commit.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git commit -am "Sixth commit"
warning: in the working copy of 'app.py', LF will be replaced by CRLF the next
[main d6eb238] Sixth commit
 1 file changed, 1 insertion(+)

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git log --oneline
d6eb238 (HEAD -> main) Sixth commit
c5cb36f fifth commit
3732bde fourth commit
a185a80 third commit
460f410 second commit
bf14698 (origin/main) first commit

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$
```

Step no:3 We can add and commit in a single line using the command: **git commit -am <commit message>**

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ vi app.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git commit -am "seventh commit"
warning: in the working copy of 'app.py', LF will be replaced by CRLF the next t
[main 2fa7232] seventh commit
 1 file changed, 4 insertions(+), 1 deletion(-)

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$
```

Using **reflog** we can see all the changes in the commit.

Now I am going to reset using soft followed by the commit id.

Soft only moves the head to previous commit.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git reflog --oneline
2fa7232 (HEAD -> main) HEAD@{0}: commit: seventh commit
d6eb238 HEAD@{1}: commit: Sixth commit
c5cb36f HEAD@{2}: commit: fifth commit
3732bde HEAD@{3}: commit: fourth commit
a185a80 HEAD@{4}: commit: third commit
460f410 HEAD@{5}: commit: second commit
bf14698 (origin/main) HEAD@{6}: Branch: renamed refs/heads/main to refs/heads/main
bf14698 (origin/main) HEAD@{8}: commit (initial): first commit

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git reset --soft 2fa7232

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git status -s

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git reflog --oneline
2fa7232 (HEAD -> main) HEAD@{0}: reset: moving to 2fa7232
2fa7232 (HEAD -> main) HEAD@{1}: commit: seventh commit
d6eb238 HEAD@{2}: commit: Sixth commit
c5cb36f HEAD@{3}: commit: fifth commit
3732bde HEAD@{4}: commit: fourth commit
a185a80 HEAD@{5}: commit: third commit
460f410 HEAD@{6}: commit: second commit
bf14698 (origin/main) HEAD@{7}: Branch: renamed refs/heads/main to refs/heads/main
bf14698 (origin/main) HEAD@{9}: commit (initial): first commit
```

Using mixed it moves the HEAD to the specified commit and updates the index to match, but does not change the working directory. This is the default mode.

It will delete the previous commit, like if we have any tracked files which has not been pushed, it will be role backed to untracked files.

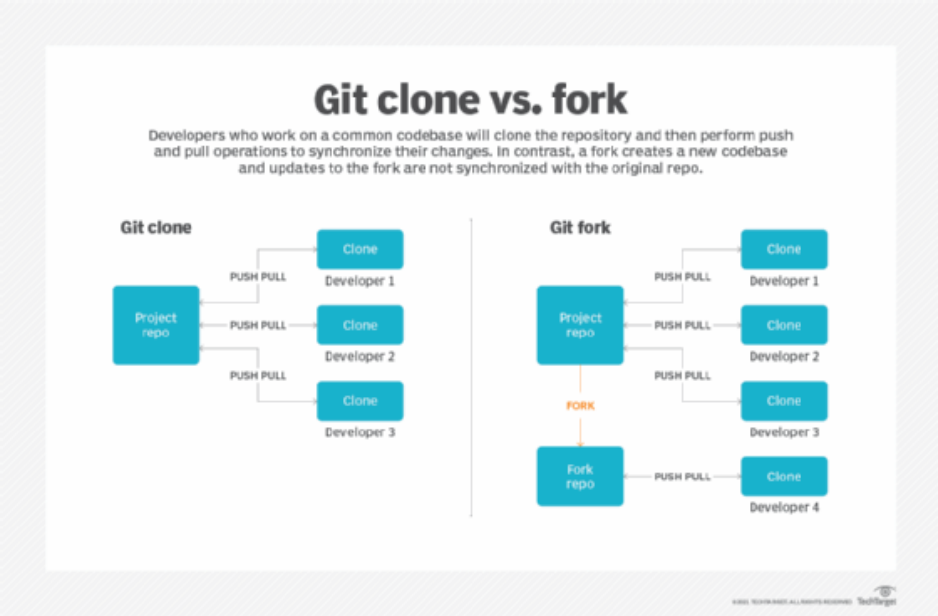
```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git reflog --oneline
2fa7232 (HEAD -> main) HEAD@{0}: reset: moving to 2fa7232
2fa7232 (HEAD -> main) HEAD@{1}: commit: seventh commit
d6eb238 HEAD@{2}: commit: Sixth commit
c5cb36f HEAD@{3}: commit: fifth commit
3732bde HEAD@{4}: commit: fourth commit
a185a80 HEAD@{5}: commit: third commit
460f410 HEAD@{6}: commit: second commit
bf14698 (origin/main) HEAD@{7}: Branch: renamed refs/heads/main to refs/heads/mai
bf14698 (origin/main) HEAD@{9}: commit (initial): first commit

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git reset --mixed 2fa7232

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/nithyafolder (main)
$ git reflog --oneline
2fa7232 (HEAD -> main) HEAD@{0}: reset: moving to 2fa7232
2fa7232 (HEAD -> main) HEAD@{1}: reset: moving to 2fa7232
2fa7232 (HEAD -> main) HEAD@{2}: commit: seventh commit
d6eb238 HEAD@{3}: commit: Sixth commit
c5cb36f HEAD@{4}: commit: fifth commit
3732bde HEAD@{5}: commit: fourth commit
a185a80 HEAD@{6}: commit: third commit
460f410 HEAD@{7}: commit: second commit
bf14698 (origin/main) HEAD@{8}: Branch: renamed refs/heads/main to refs/heads/mai
bf14698 (origin/main) HEAD@{10}: commit (initial): first commit
```

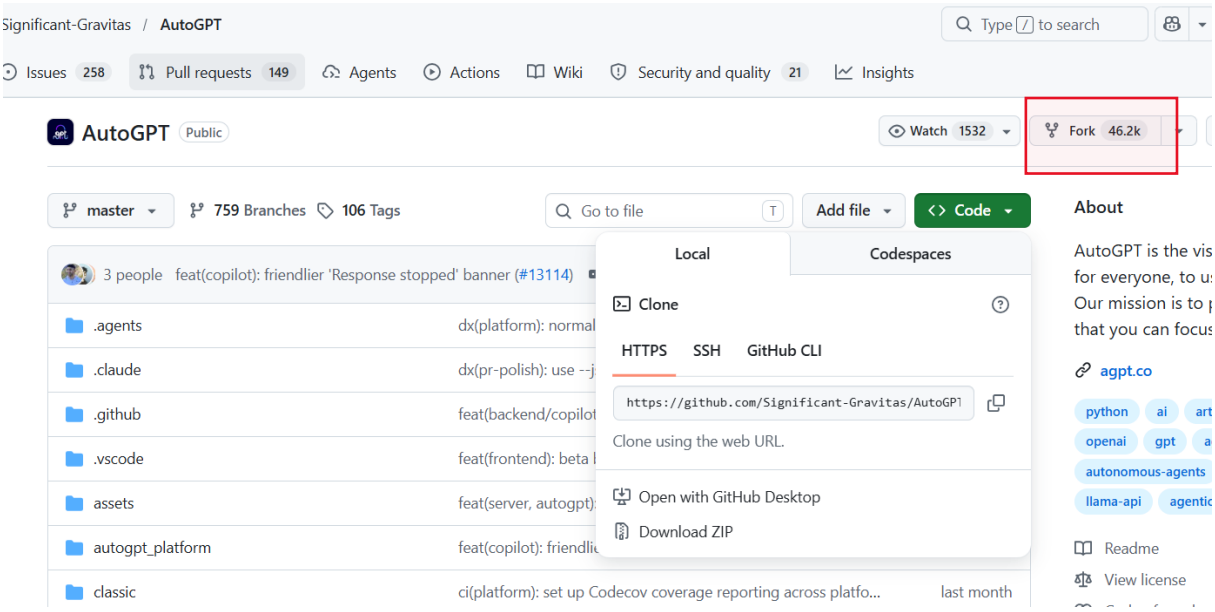
Forking a Repository on GitHub

Forking a repository on GitHub allows you to create a personal copy of someone else's project. This is useful for proposing changes, experimenting with new ideas, or using someone else's project as a starting point for your own work. Here's a step-by-step guide on how to fork a repository:

Steps to Fork a Repository

1. **Navigate to the Repository**: Go to the repository you want to fork on GitHub. You can do this by searching for the repository or navigating directly to its URL.
2. **Click the Fork Button**: In the top-right corner of the repository page, you'll see a "Fork" button. Click on it. This will create a copy of the repository under your GitHub account.
3. **Select an Owner**: If you have multiple organizations or accounts, you can select where you want to fork the repository. Choose the appropriate owner from the dropdown menu.
4. **Name and Describe Your Fork**: By default, the forked repository will have the same name as the original. You can change the name and add a description if you want to distinguish it from the original.
5. **Create the Fork**: Click on the "Create fork" button. GitHub will create a copy of the repository under your account. This process may take a few seconds.





Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project. View existing forks.

Required fields are marked with an asterisk (*).

Owner * NITHYAVEL04 / **Repository name** * AutoGPT

AutoGPT is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description

AutoGPT is the vision of accessible AI for everyone, to use and to build on. Our mission is to provide the tools,

152 / 350 characters

Copy the `master` branch only

Contribute back to Significant-Gravitas/AutoGPT by adding your own branch. Learn more.

You are creating a fork in your personal account.

Create fork